
Spring Security

Authorization Exercise

Endpoint-level and method-level authorization

Scenario: Order Management API

You will secure a small Spring application by combining request authorization rules with method-level authorization. The exercise progresses from URL matching and roles to authorities, custom authorization logic, and method parameters.

Learning objectives

- Configure authorization rules using `SecurityFilterChain`.
- Distinguish between roles and authorities.
- Enable and apply method security with `@PreAuthorize`.
- Use the authenticated user inside authorization expressions.
- Delegate authorization decisions to a custom Spring bean.
- Explain why endpoint-level and method-level authorization solve different problems.

Assumption: Authentication is already configured. Authenticated users are assigned the roles and authorities described in the exercise.

Application Scenario

You are developing an internal **Order Management** application. The application exposes the following HTTP endpoints.

Endpoint	Method	Description
/api/products	GET	List available products
/api/orders	POST	Create an order
/api/orders/{id}	GET	View an order
/api/orders/{id}	DELETE	Delete an order
/api/admin/reports	GET	View sales reports

The application has three types of users:

- CUSTOMER
- MANAGER
- ADMIN

Important: Focus on *authorization*. You do not need to implement login, password storage, JWT validation, or user persistence for this exercise.

Tasks

Requirement 1 - Endpoint-level authorization

Configure Spring Security using a `SecurityFilterChain` so that:

- GET `/api/products` is accessible without authentication.
- `/api/admin/**` can only be accessed by users having the `ADMIN` role.
- Creating orders with POST `/api/orders` requires authentication.
- All other `/api/**` requests require authentication.
- Any request outside these rules should be denied.

Implement the rules using `authorizeHttpRequests()` and request matchers.

Requirement 2 - Enable method security

Enable Spring Security method-level authorization in the application.

Identify and add the appropriate Spring Security annotation required to activate method security.

Requirement 3 - Manager-only service operation

The application contains the following service operation:

```
@Service
public class OrderService {

    public void cancelAnyOrder(Long orderId) {
        // ...
    }
}
```

Only users with the **MANAGER** or **ADMIN** role should be allowed to invoke `cancelAnyOrder()`. Add the appropriate method-security annotation.

Requirement 4 - Authority-based authorization

Users may also have the following authority:

```
ORDER_REFUND
```

Consider this service method:

```
public void refundOrder(Long orderId) {
    // ...
}
```

Configure method-level security so that this method can only be called by users having the **ORDER_REFUND** authority.

Constraint: The solution must not depend on a role.

Requirement 5 - Accessing the authenticated user

Consider the following service method:

```
public Order getOrder(Long orderId) {
    // ...
}
```

The application already contains this security component:

```
@Component
public class OrderSecurity {

    public boolean isOwner(
        Long orderId,
        Authentication authentication) {
        // returns true if the authenticated user owns the order
    }
}
```

Configure `getOrder()` so that it can be invoked when either:

- the authenticated user owns the order, or
- the authenticated user has the **ADMIN** role.

Call the custom security component from a `@PreAuthorize` expression.

Requirement 6 - Endpoint security vs. method security

The controller exposes:

```
@GetMapping("/api/orders/{id}")
public Order getOrder(@PathVariable Long id) {
    return orderService.getOrder(id);
}
```

Configure the endpoint so that `/api/orders/{id}` requires authentication at the HTTP level. Keep the ownership/administrator rule at the **service-method level**.

Then answer the following question in 2-4 sentences:

Why do these two authorization checks solve different problems?

Bonus Challenge

Status update rule using method parameters

Add this method to the service:

```
public void updateOrderStatus(Long orderId, String status) {
    // ...
}
```

Apply the following rules:

- MANAGER can call the method.
- ADMIN can call the method.
- A manager may set the status to PROCESSING or SHIPPED.
- Only an administrator may set the status to CANCELLED.

Implement the entire authorization rule using a single `@PreAuthorize` SpEL expression.

Expected Progression

The exercise intentionally builds authorization concepts in this order:

Request matchers → Roles → Authorities → Method security
→ Authentication object → Custom authorization bean → Method parameters

What to submit

Submit the following:

- The security configuration class containing the endpoint-level rules.
- The service class with the required method-security annotations.
- The annotation/configuration used to enable method security.
- Your short explanation for Requirement 6.
- Optional: the Bonus Challenge solution.